

四電一丙 數位邏輯實習

第 2 次實習作業報告

組員 1 學號:B812111001 姓名: 張博翔

組員 2 學號:114310339 姓名:陳映融

組員 1 於此作業中的貢獻:解題 digital quartus 拍照 (作業一、二) 作業報告(二次編排、修改與美編)

組員 2 於此作業中的貢獻:解題 digital 拍照 (作業一、二) 作業報告(初步編排、複審內容)

I. 實驗目的

A. 作業 1

設計一個完整的七段解碼器，運用四位元加減法器，將輸出接上七段顯示器，並連接偵測溢位(overflow)功能電路，熟悉模組化電路與 IO 使用之技巧。

B. 作業 2

設計 4-bit 前瞻進位加法器(Carry-Lookahead adder)，將此電路與作業題 1 設計之四位元加法器串聯，並利用 Single Gate Simulation 與 Ripple Carry adder 比較執行速度，證明電路功能。

II. 設計程序

A. 作業 1

1. 依據全加器的邏輯概念建立真值表，再轉換為卡諾圖。

表 1. Full adder 真值表

X	Y	C_{in}	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$X \setminus YC_{in}$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

圖 1. Sum bit 卡諾圖

$X \setminus YC_{in}$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

圖 2. Cout bit 卡諾圖

2. 參考卡諾圖將布林函式化簡，使用 Digital 完成連接進行電路模擬，測試不同輸入組合，確認輸出結果符合原先表示式後將其作為模組。

$$\begin{aligned}
 Sum &= X'Y'C_{in} + X'YC'_{in} + XY'C'_{in} + XYC_{in} \\
 &= X'(Y'C_{in} + YC'_{in}) + X(Y'C'_{in} + YC_{in}) \\
 &= X'(Y \oplus C_{in}) + X(Y \oplus C_{in})' = X \oplus Y \oplus C_{in}
 \end{aligned}$$

圖 3. Sum bit 表示式

$$\begin{aligned}
 C_{out} &= X'YC_{in} + XY'C_{in} + X'YC'_{in} + XYC_{in} \\
 &= (X'YC_{in} + X'YC'_{in}) + (XY'C_{in} + XYC_{in}) + (XY'C'_{in} + XYC_{in}) \\
 &= YC_{in} + XC_{in} + XY
 \end{aligned}$$

圖 4. Cout bit 表示式

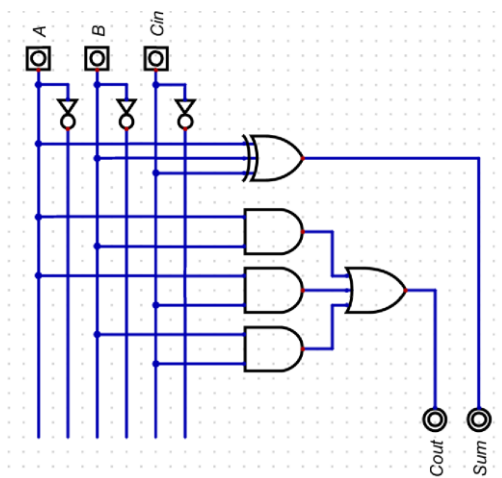


圖 5. 1-bit Full adder 邏輯電路

3. 將模組串聯合成 4-bit Ripple Carry Adder，為了實現加減法器(Adder-Subtractor)，需讓 Cin 與其中一組數值進行 XOR 運算，當 Cin=0，兩數值直接相加；當 Cin=1，則與 Cin XOR 運算的數值會取 1 補數，加上 Cin 取 2 補數，達成減法效果，確認輸出結果符合預期後將其作為模組。

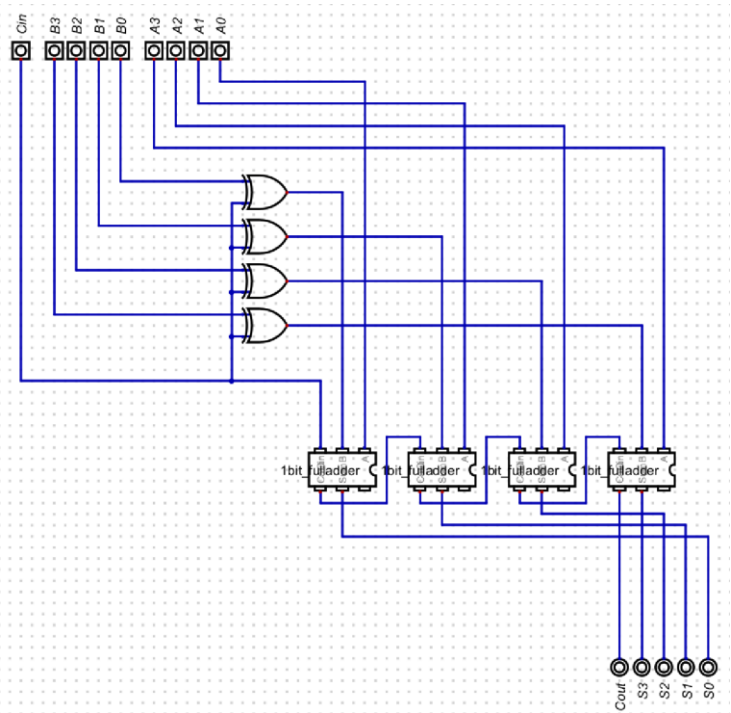


圖 6. 4-bit Adder-Subtractor 邏輯電路

4. 利用 Digital 的真值表生成 SOP 電路功能產出七段顯示器解碼電路，由於輸出為 4-bit 有號數，因此真值表範圍應包含 0 ~ 7、-1 ~ -8，確認後作為模組。

表 2. 七段顯示器真值表

S3	S2	S1	S0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	0	0
1	0	1	0	1	0	1	1	1	1	1
1	0	1	1	1	0	1	1	0	1	1
1	1	0	0	0	1	1	0	0	1	1
1	1	0	1	1	1	1	1	0	0	1
1	1	1	0	1	1	0	1	1	0	1
1	1	1	1	0	1	1	0	0	0	0

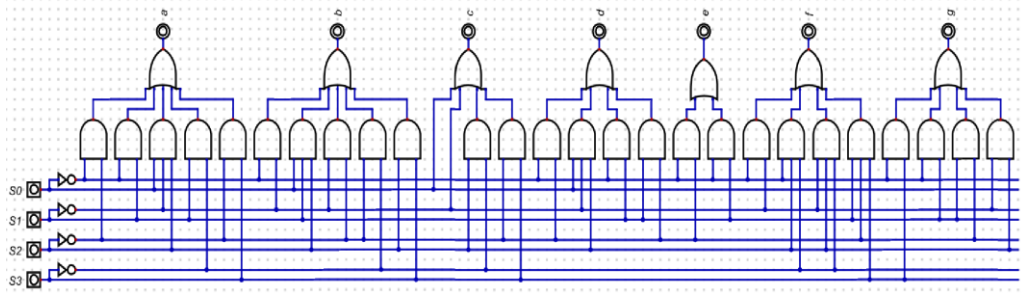


圖 7. 七段顯示器邏輯電路

- 開啟 Embedded Circuit 將各模組串接，並且運用兩組數值 MSB 與 Sum MSB 判斷溢位，若兩組數值 MSB 相同但相加後 Sum MSB 變號則發生溢位。運用 Digital 內建的七段顯示器搭配進行模擬(由於 FPGA 的七段顯示器為共陽型，需要將 Digital 設定為 Common Connection: Anode)，測試不同輸入組合，確認輸出結果符合原先表示式後產出 VHDL 檔案。

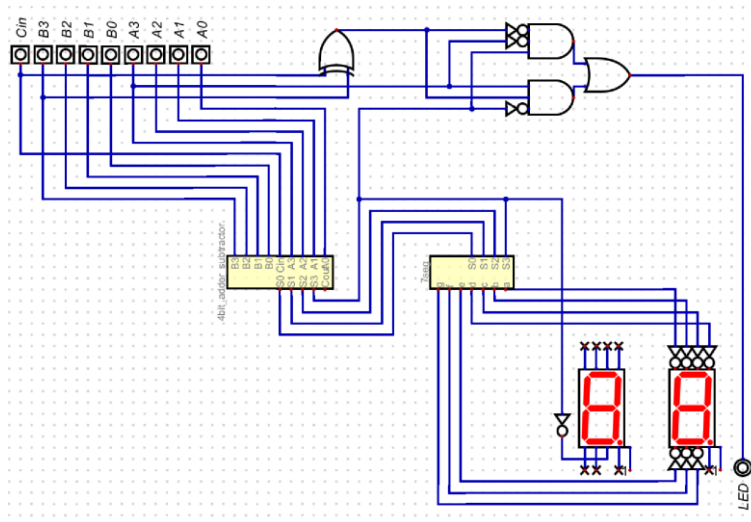


圖 8. 七段解碼器模擬電路圖

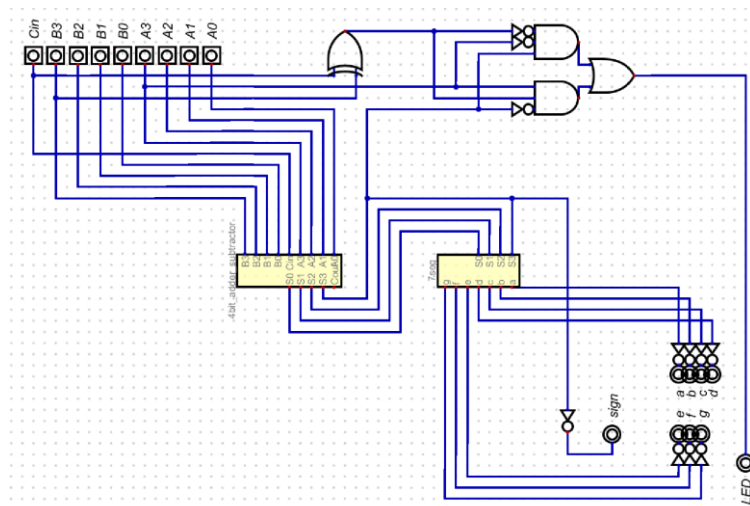


圖 9. 七段解碼器輸出電路圖(VHDL 產出)

- 開啟 Quartus 建立專案，導入 VHDL，修改內容檔名後進行第一次編譯，待變數讀入後定義腳位並再次編譯，完成後進行 FPGA 燒錄，並進行結果測試。

B. 作業 2

- 參考前瞻進位加法器 Carry 的邏輯表示式，運用 Digital 中的 Expression 功能，分別輸入 C1~C4 邏輯表示式(需展開到 C0 否則電路需要等待前一個結果)，得到 Carry-Lookahead 邏輯電路，並作為模組。

$$P_i = A_i \oplus B_i$$

$$G_i = A_i \cdot B_i$$

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot C_1$$

$$= G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0)$$

$$= \mathbf{G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0}$$

$$C_3 = G_2 + P_2 \cdot C_2$$

$$= G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0)$$

$$= \mathbf{G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0}$$

$$C_4 = G_3 + P_3 \cdot C_3$$

$$= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0)$$

$$= \mathbf{G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0}$$

圖 10. Carry-Lookahead 輸入、輸出表示式

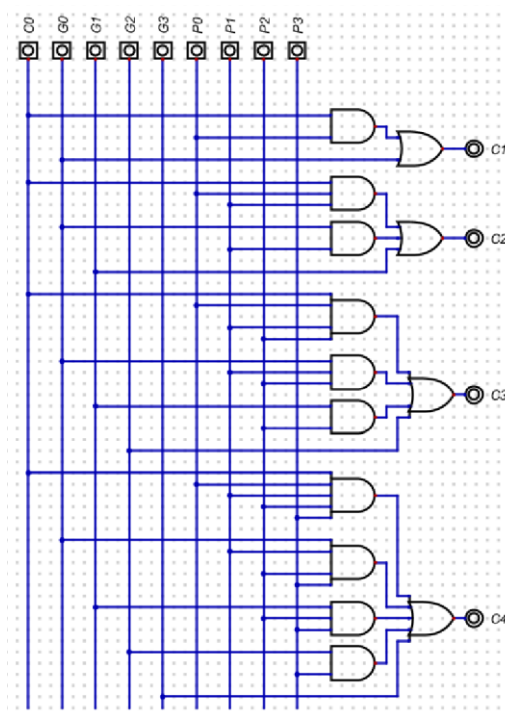


圖 11. Carry-Lookahead 邏輯電路

- 由於 Carry-Lookahead 輸入需要為 G (Generate)、P (Propagate)、S (Sum)，因此需要參考圖 10.之表示式，修正 1-bit Full adder 模組。

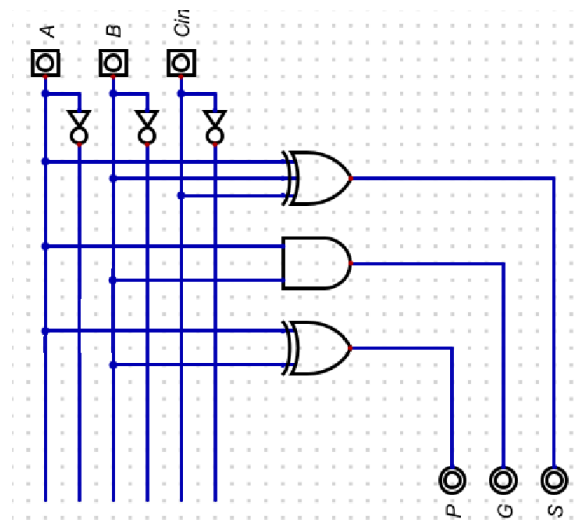


圖 12. 調整輸出之 1-bit Full adder 邏輯電路

- 開啟 Embedded Circuit 將原先的 1-bit Full adder、Carry-Lookahead circuit 串接，組合出 Ripple Carry Adder 與 Carry-Lookahead adder，運用 Single Gate Step Mode 執行模擬，判斷兩者執行差異並記錄。

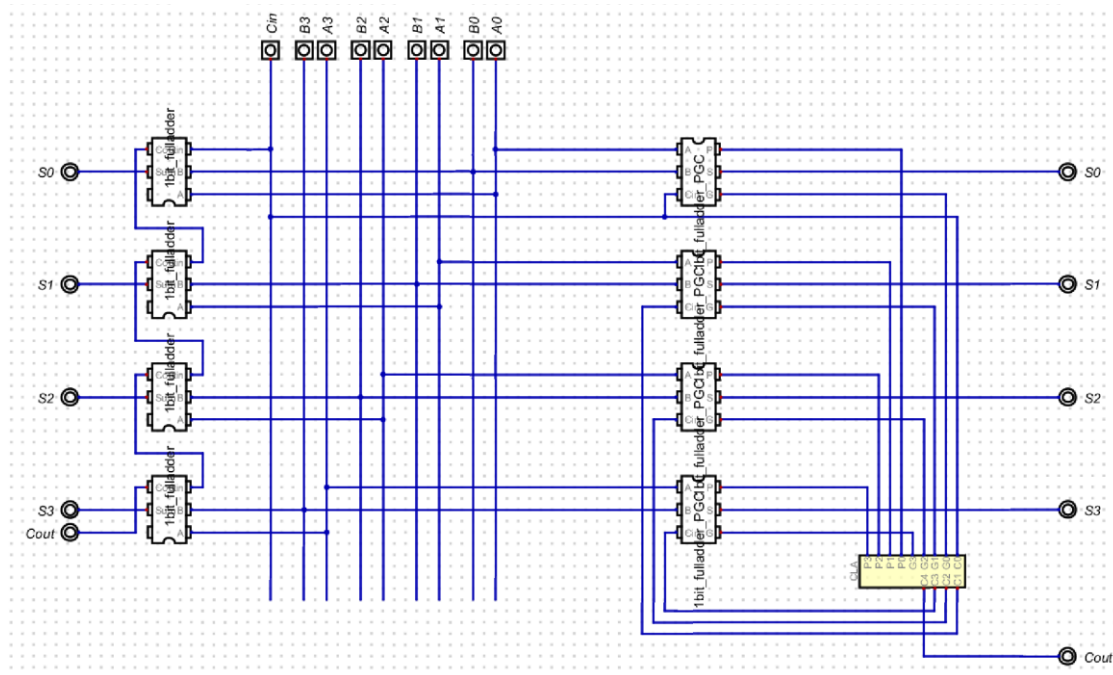


圖 13. RCA(左) / CLA(右) 邏輯電路

III. 模擬驗證

A. 作業 1

七段解碼電路：Input A($A_3A_2A_1A_0$)、B($B_3B_2B_1B_0$)、Control C(C_{in})，Output a、b、c、d、e、f、g (以下為方便判斷輸出以計算結果代替)

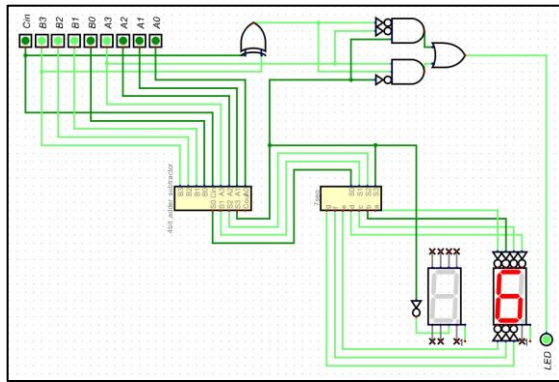


圖 14. Input:1000(A)+1110(B) / Output:(1)0110
Overflow

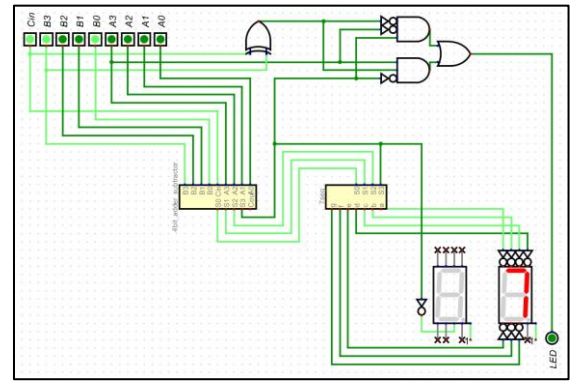


圖 15. Input:0000(A)-1001(B) / Output:0111

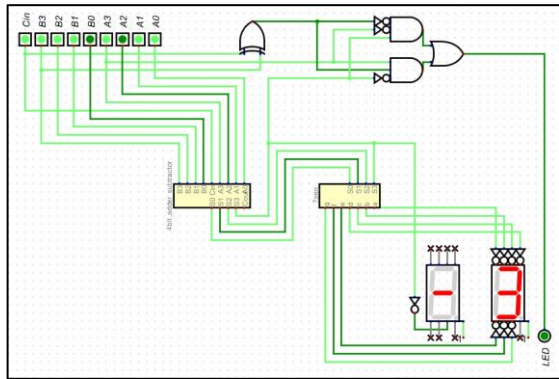


圖 16. Input:1011(A)-1110(B) / Output: 1101

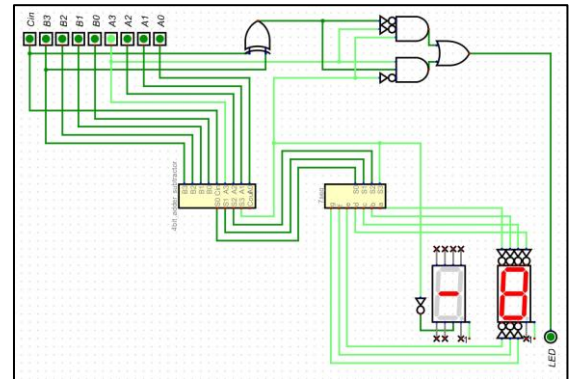


圖 17. Input:1000(A)+0000(B) / Output:1000

B. 作業 2

Carry-Lookahead adder 與 Ripple Carry adder 速度比較：Calculate：0011+0101 Result：1000

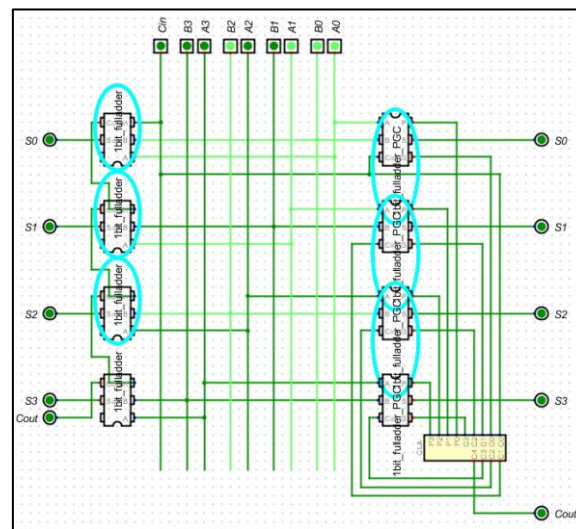
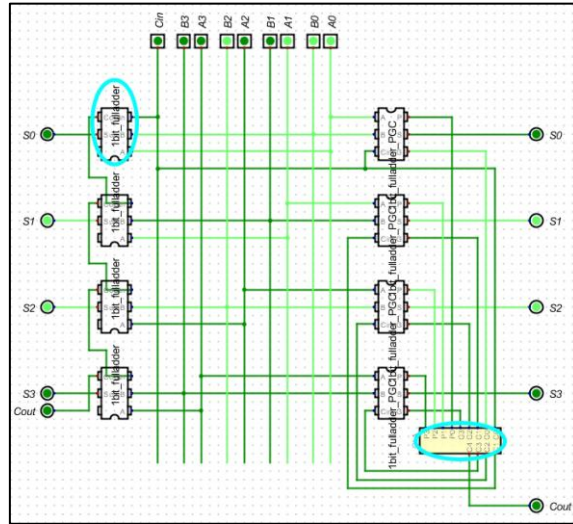
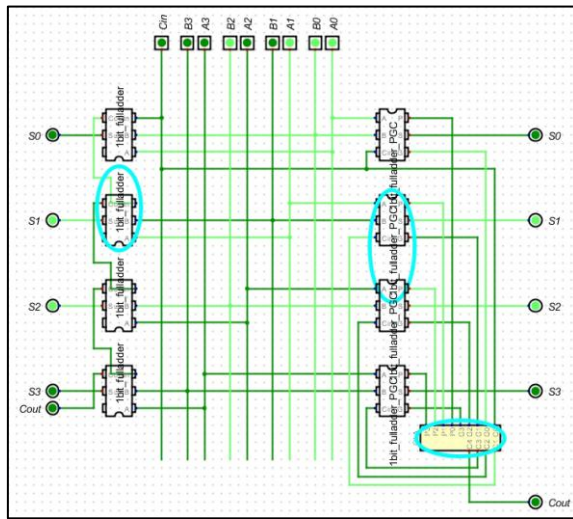


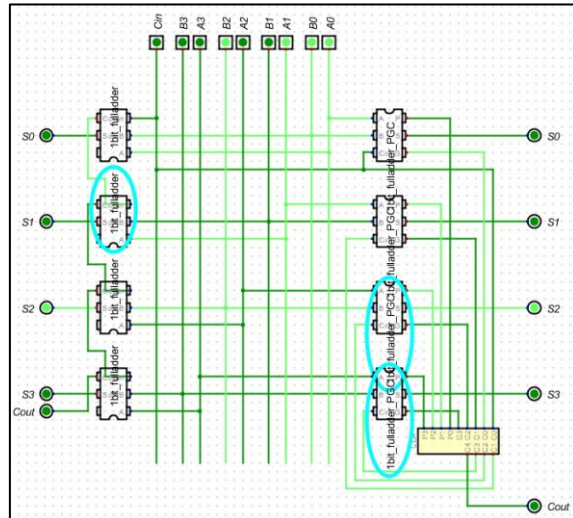
圖 18. RCA(左) : (0)0000 / CLA(右) : (0)0000 Step1



19. RCA : (0)0110 / CLA : (0)0110 Step2



20. RCA : (0)0110 / CLA : (0)0110 Step3



21. RCA : (0)0100 / CLA : (0)0100 Step4

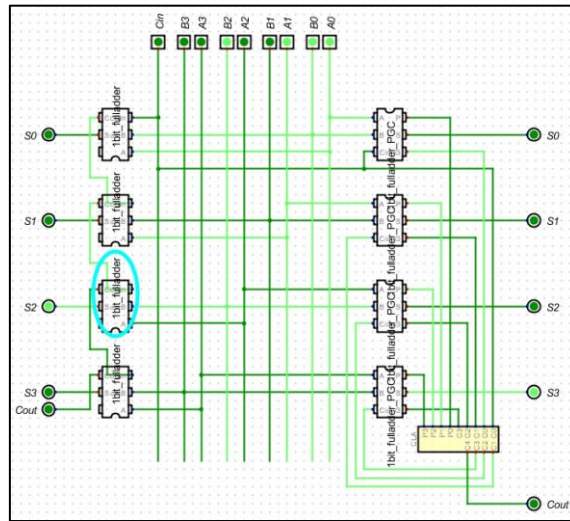


圖 22. RCA : (0)0100 / CLA : (0)1000 Step5

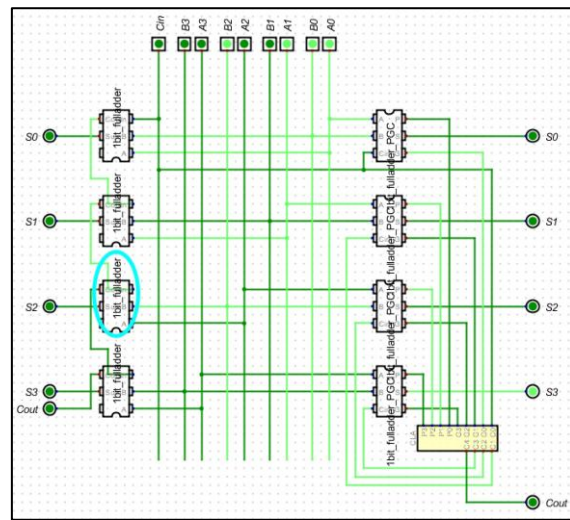


圖 23. RCA : (0)0000 / CLA : (0)1000 Step6

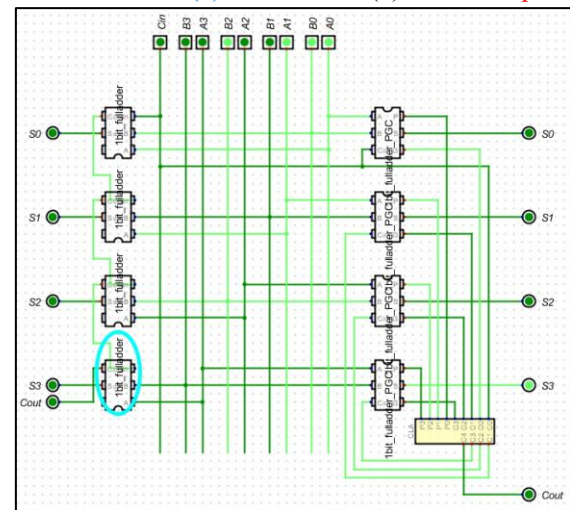


圖 24. RCA : (0)0000 / CLA : (0)1000 Step7

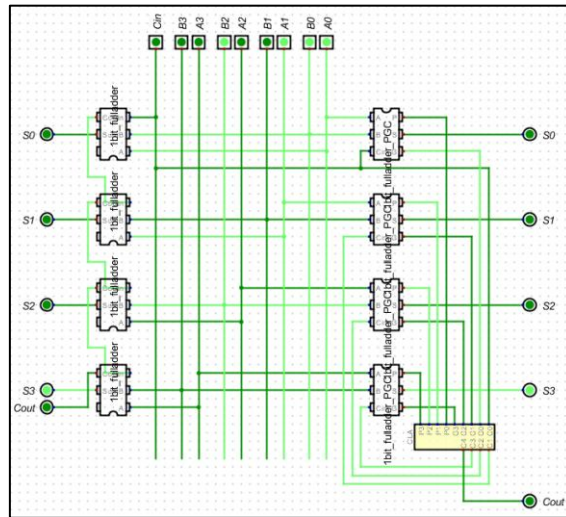


圖 25. RCA : (0)1000 / CLA : (0)1000 Step8

IV. 實驗結果與成果討論

A. 作業 1

七段解碼電路：Input A($A_3A_2A_1A_0$)、B($B_3B_2B_1B_0$)、Control C(C_{in})，Output a、b、c、d、e、f、g (以下為方便判斷輸出以計算結果代替)。(以下結果均符合模擬預測)

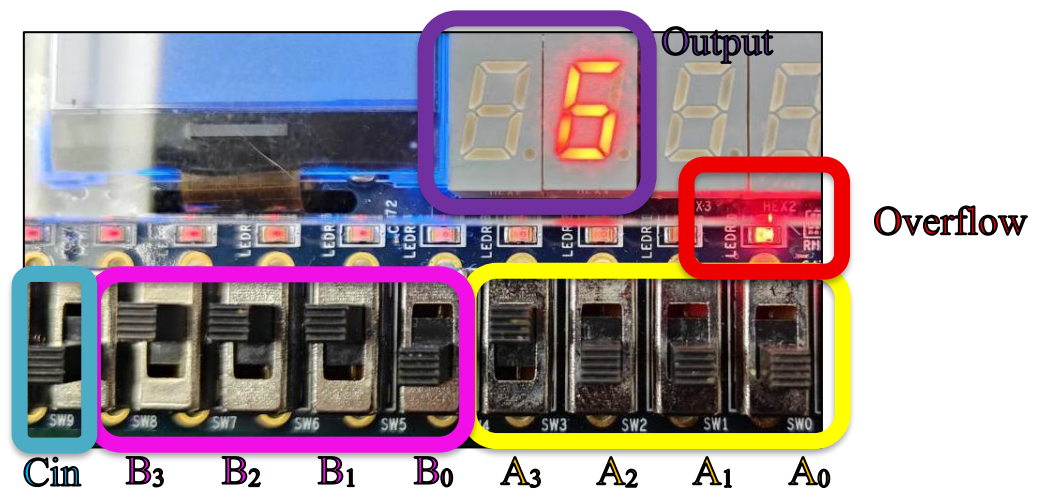


圖 26. Input:1000(A)+1110(B) / Output:(1)0110 overflow

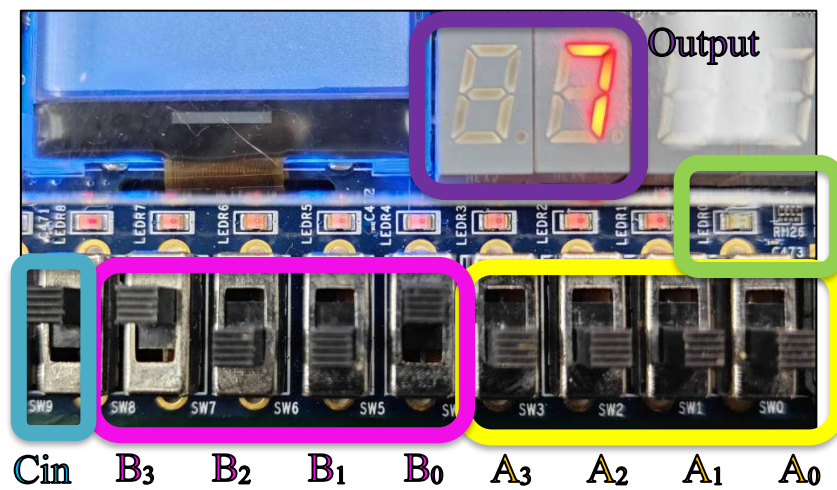


圖 27. Input:0000(A)-1001(B) / Output:0111

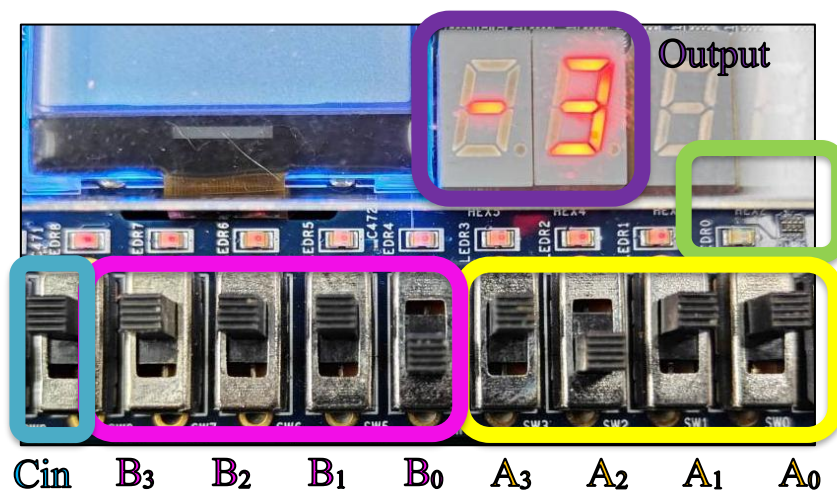


圖 28. Input:1011(A)-1110(B) / Output: 1101

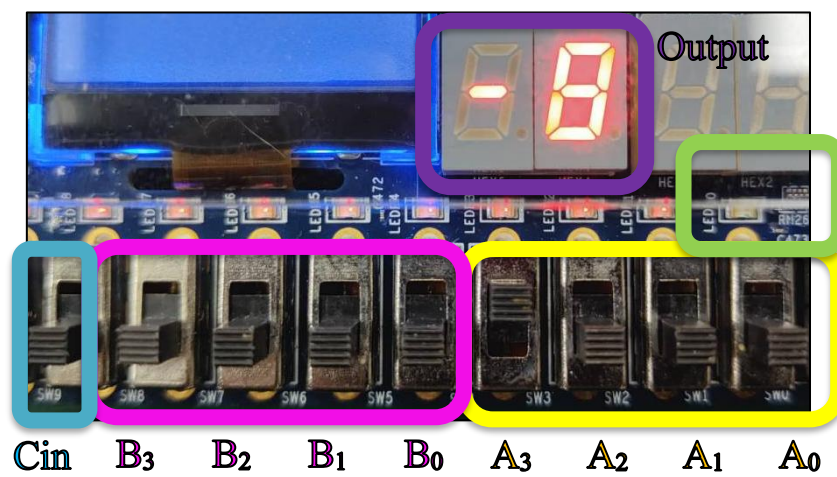


圖 29. Input:1000(A)+0000(B) / Output:1000

B. 作業 2

Carry-Lookahead adder 與 Ripple Carry adder 速度比較：Calculate : $0011+0101$ Result : 1000 。(以下結果均符合模擬預測)

可以觀察到輸出在 Step 8 (圖.25)時 Carry-Lookahead adder 與 Ripple Carry adder 皆為 1000，無進位，符合預期輸出。兩者速度差異在 Step 5 (圖.22)最為明顯，此時 CLA 已完成整體運算，而 RCA 仍處於第三級全加器的進位傳遞階段。實驗結果證明，CLA 領先 RCA 四個 Step，成功驗證了 CLA 在多位元運算中，能透過平行產生進位的邏輯克服 RCA 的進位傳遞延遲，提升運算速度。

V. 實驗心得

組員 1：作業一的溢位偵測器設計還有改進空間，可以調整模組輸出，運用 C_3 與 C_4 做 XOR，簡化電路設計。另外實驗與概念的熟練度都還待加強，雖然說這一次要作的內容不算少，但釐清概念後難度並不算太高，還需持續練習。

組員 2：兩台電腦之間共用子電路略為麻煩要一直用 USB 傳檔案，要更改錯誤更麻煩，下次力求一次到位。

參考文獻

- [1] DLE 03 數位邏輯實習 全加減法器、七段顯示器